CS 2840 Fall 2025 Scribes: Mary Letey
08/10/2025 Based on notes by: Mary Letey

Lecture 11: Optimal Transport as a Loss Function

Recap

Last time we saw

- Different ways to compare distributions: Horizontally (globally moving mass), Vertically (locally by pointwise density ratios), and Functionally (compare by expectation of witness functions)
- Vertical comparison relies on using f-divergences
- Functional comparison with IPMs

Now we move on to a new topic:

OPTIMAL TRANSPORT AS A LOSS FUNCTION

Statistical divergences are often used as a loss function in **parameter estimation** problems, with wide array of applications: synthesis, image segmentation, inverse problems, and shape matching. This is particularly useful when we understand the data properties well or are working with low intrinsic dimensional data. OT has many nice properties that motivate its use as a loss function, which we will see in the following lecture. The main challenge in using OT as a loss is the feasibility of training and fast differentiation needed. We will cover

- Variational objectives
- Differentiability of OT
- Using autodifferentiation effectively
- Using minibatching

OT as a Variational Objective

A classic setup is that we have a true distribution β we wish to learn using a parametric model class $\{\alpha_\theta \text{ for } \theta \in \Theta\}$. Here θ can encode parameters of a prior (empirical measures) or parameters of a map i.e. a neural net (continuous measures). We seek to fit a parametric model distribution α_θ to a target distribution β from a model class $\{\alpha_\theta : \theta \in \Theta\}$.

$$\min_{\theta\in\Theta} \mathcal{L}(\theta) \overset{\text{def}}{=} \mathcal{L}(\alpha_\theta; \beta), \qquad \text{with} \quad \mathcal{L}(\alpha_\theta, \beta) = \mathrm{OT}_c(\alpha_\theta, \beta).$$

This setup extends to barycenters, dictionary learning, and related composite objectives.

Examples.

- **Empirical measures (particles).** Encode locations x_i and/or weights a_i directly, or parameters of a prior that generates them.
- **Continuous measures (maps).** Let $\alpha_\theta = (g_\theta)_\sharp \zeta$ be the pushforward of a simple base ζ (e.g. Gaussian) by a parameterised map g_θ (e.g. a neural network).

What Makes a Good ML Loss?

A useful loss should be:

1. **Optimisable:** differentiable (preferably smooth), informative gradients, and amenable to principled regularisation.
2. **Practical & Scalable:** computationally efficient, mini-batch/SGD friendly, and easy to integrate in modern pipelines.
3. **Aligned with the task:** robust to noise/outliers/misspecification and reflective of the domain's structure.

Differentiability of (E)OT

Classic discrete OT solves a linear program and is not differentiable in general: the optimal plan $\pi^\star$ may be non-unique, and small perturbations to (α, β) can change the active set. Differentiability typically requires uniqueness/stability of the coupling and strict convexity of the cost; in practice this is often addressed by **regularisation**.

Recall Kantorovich's Problem

$$\min_{\pi \in \Pi(\alpha,\beta)} \langle \pi, C \rangle - \varepsilon H(\pi)$$

is strictly convex, so there is a unique minimiser. The object is differentiabile wrt π. The unique minimiser π_ε^* varies smoothly on α, β, c. The dual problem

$$\max_{f,g} \langle f, \alpha \rangle + \langle g, \beta \rangle - \varepsilon \langle e^{(f \oplus g - c)/\varepsilon} - 1, \alpha \otimes \beta \rangle$$

is smooth and concave i.e. differentiable wrt f, g. Note that in this problem, optimal potentials vary smoothly!

Remark. In most applications we only have samples, so we estimate OT from empirical measures.

- **Eulerian (grid / histogram):** learn $a_i(\theta)$ on a fixed support $\{x_i\}$. Here we need $\partial \, \mathrm{OT}/\partial a_i$ and possibly $\partial \, \mathrm{OT}/\partial C$.
- **Lagrangian (particles):** fix weights (often $a_i = \frac{1}{n}$) and learn support $x_i(\theta)$. In this case we need $\partial \, \mathrm{OT}/\partial x_i$ (via $\partial \, \mathrm{OT}/\partial C$).

Derivatives wrt Histograms (Eulerian)

For $\varepsilon > 0$, OT_ε is convex and differentiable in (a, b). If (f, g) solves the dual, centered to zero mean, then

$$\nabla_a \, \mathrm{OT}_\varepsilon(a, b) = f, \qquad \nabla_b \, \mathrm{OT}_\varepsilon(a, b) = g, \qquad \sum_i f_i = \sum_j g_j = 0.$$

By the chain rule, if $a = a(\theta)$, then

$$\nabla_\theta \mathcal{L}(\theta) = [D_\theta a(\theta)]^\top f.$$

Derivatives wrt Cost and Support (i.e. Lagrangian)

For fixed a, b and $\varepsilon > 0$, OT_ε is smooth and *concave* in C with gradient

$$\nabla_C \, \mathrm{OT}_\varepsilon(a, b) \; = \; \pi_\varepsilon^\star.$$

If $C_{ij} = c(x_i, y_j)$ with learnable x_i, then by the chain rule

$$\nabla_{x_i} \, \mathrm{OT}_\varepsilon(a, b) = \sum_{j=1}^{m} \pi_{ij}^\star \, \nabla_x c(x_i, y_j), \qquad \nabla_\theta \mathcal{L}(\theta) = [D_\theta X(\theta)]^\top \nabla_X \, \mathrm{OT}_\varepsilon(a, b).$$

We can use this to minimise $\mathcal{L}$ wrt θ by gradient descent.

Pop quiz: squared cost gradient – Let $c(x,y) = \|x-y\|_2^2$. Then

$$\nabla_{x_i} \operatorname{OT}_\varepsilon(a,b) \;=\; 2\left(a_i x_i - \sum_{j=1}^{m} \pi^\star_{ij} y_j\right) \;=\; \lambda\big(I - T_{\text{bary}}\big)(x_i),$$

i.e. proportional to the deviation from the $\pi^\star$-barycentric projection.

General Differentiability (Measure Viewpoint)

For probability measures $\alpha, \beta \in \mathcal{P}(\mathcal{X})$, $\operatorname{OT}_\varepsilon(\alpha,\beta)$ is weak-* continuous and differentiable in either argument. Viewing $\operatorname{OT}_\varepsilon$ as a functional on $\mathcal{P}(\mathcal{X})$, the first variation is given by the dual potentials:

$$\nabla_\alpha \operatorname{OT}_\varepsilon(\alpha,\beta) = f, \qquad \nabla_\beta \operatorname{OT}_\varepsilon(\alpha,\beta) = g,$$

where $f = T(\beta, g)$ and $g = T(\alpha, f)$ with the (soft-min) Sinkhorn map

$$T(\alpha,f)(y) \;=\; -\varepsilon \log \int_{\mathcal{X}} \exp\left(\frac{-c(x,y)+f(x)}{\varepsilon}\right) \mathrm{d}\alpha(x).$$

Now we move on to the practical consideration of actually computing these things in a scalable way for ML, i.e. autodifferentiation!

EOT derivatives using Autodifferentiation

Closed-form gradients require exact EOT solutions (f,g) or $\pi^\star$, but in practice we compute them approximately (e.g. a fixed number of Sinkhorn iterations). Running to full convergence can be impractical and Jacobians can be large. A flexible alternative is to backpropagate *through* the Sinkhorn iterations using automatic differentiation.

Write $K = \exp(-C/\varepsilon)$ and run L Sinkhorn updates

$$u^{\ell+1} = \frac{a}{K v^\ell}, \qquad v^{\ell+1} = \frac{b}{K^\top u^{\ell+1}}, \qquad \ell = 0, \dots, L-1,$$

yielding $\pi^{(L)} = \operatorname{Diag}(u^{(L)})\, K \operatorname{Diag}(v^{(L)})$ and an estimate of $\operatorname{OT}_\varepsilon$. AutoDiff yields gradients w.r.t. any parameters of $a(\theta)$, $X(\theta)$, or $c_\varphi(\cdot,\cdot)$ and leverages efficient GPU kernels (e.g. `logsumexp`) already present in deep learning libraries.

This formulation has the following **Advantages**

- Avoids constructing full Jacobians (uses vJP/JvP).
- GPU-friendly and batched; works with arbitrary parameterisations (including deep nets).
- Tunable accuracy–speed trade-off via L and detach tricks.

We will see one of these tricks explicitly now.

Stop-Gradient Trick

At convergence, $\nabla_a \operatorname{OT}_\varepsilon(a,b) = f$. One can use the last iterate $f^{(L)}$ as a surrogate and *stop* backprop through the Sinkhorn loop to reduce memory:

- run Sinkhorn in a no-gradient context; or
- retroactively cut the graph with `f.detach()`.

Mini-Batch Estimation

Computing OT on full datasets is expensive. Use mini-batch empirical measures

$$\hat{\alpha}_B = \frac{1}{B}\sum_{i=1}^{B}\delta_{x_i}, \qquad \hat{\beta}_B = \frac{1}{B}\sum_{j=1}^{B}\delta_{y_j}, \qquad \widehat{\mathcal{L}}(\theta) = \mathrm{OT}_\varepsilon\big(\hat{\alpha}_B(\theta), \hat{\beta}_B\big).$$

Bias can be controlled by sufficient batch size B, appropriate ε, and averaging across batches. In expectation, the batched objective upper-bounds the full-data OT, and the bound can be tightened with thoughtful batch pairing strategies.

Applications

Generative Modeling. Train a generator g_θ by minimising an OT-based loss between generated and real samples. Advantage over GAN/WGAN: no adversary; just a generator plus a differentiable loss layer (EOT).

Knowledge Distillation. minimise an OT/Sinkhorn divergence between teacher and student logit distributions across classes; empirically improves compression for large models.